

 RAG ARCHITECTURES

5 RAG Architectures – Which One Do You Need?

Naive RAG gets you started.
The other 4 keep you **in production.**



AI Coach John

FOLLOW



 Repost

One Architecture **Doesn't Fit All**

THE PROBLEM

Naive RAG works for demos. But documents have relationships. Queries need verification. Some questions need reasoning chains. Others need fresh web data.

One pattern can't handle all these - each architecture was built to solve a specific failure mode.



Naive

MVP & demos



CRAG

Unreliable docs



Adaptive

Mixed queries



Graph

Relationships



Agentic

Complex tasks



Each architecture was invented because the previous one failed in a specific real-world scenario. Know the failure, know when to upgrade.



AI Coach John

FOLLOW



↻ Repost

Naive RAG - The Starting Point

HOW IT WORKS

- > Chunk documents into fixed-size pieces
- > Embed each chunk with a text embedding model
- > Store in vector database
- > At query time: embed query → similarity search → top-k chunks → LLM

✓ WHEN TO USE

- > MVP and proof of concept
- > High-quality, consistent documents
- > Simple factual Q&A
- > Speed to market > accuracy

✗ FAILURE MODES

- > Retrieves wrong chunks for ambiguous queries
- > No fallback when retrieval fails
- > Can't handle multi-hop questions

≡ PIPELINE

Documents

↓ chunk + embed

Vector Store

↓

User Query

↓ embed + search

Top-K Chunks

↓ as context

LLM

↓

Response

</> QUICK START

```
from langchain_chroma import Chroma
from langchain_openai import (
    OpenAIEmbeddings
)

db = Chroma.from_documents(
    docs,
    OpenAIEmbeddings()
)
retriever = db.as_retriever(
    k=5
)
```



CRAG - Self-Correcting Retrieval

THE PROBLEM IT SOLVES

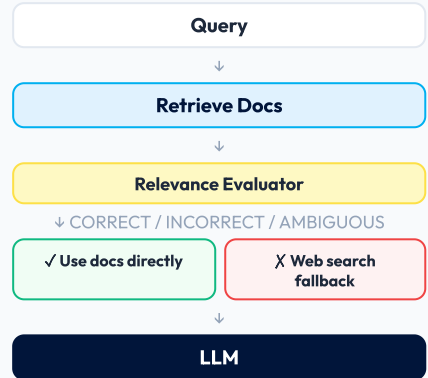
Scenario: Internal legal Q&A. User asks about a merger clause. Naive RAG retrieves an old contract version. LLM confidently answers - from stale data.

CRAG adds a **relevance evaluator** after retrieval: is this retrieved content actually useful?

✓ WHEN TO USE

- Heterogeneous document collections
- Documents of varying quality
- When hallucination rate is too high

CRAG PIPELINE



</> EVALUATOR LOGIC

RELEVANT → use retrieved chunks as-is
 IRRELEVANT → fallback to web search (Tavily/Serper)
 AMBIGUOUS → use both, let LLM weigh confidence



AI Coach John

FOLLOW



↺ Repost

CRAG Implementation with LangGraph

</> LANGGRAPH CRAG

```
from langgraph.graph import (
    StateGraph, END
)

def retrieve(state):
    docs = retriever.get_relevant_documents(
        state["query"]
    )
    return {"documents": docs}

def grade_documents(state):
    """LLM-as-judge: relevant or not?"""
    score = grader.invoke({
        "question": state["query"],
        "document": state["documents"][0]
    })
    if score.binary_score == "yes":
        return {"grade": "relevant"}
    else:
        return {"grade": "irrelevant"}

def web_search(state):
    results = tavily.search(state["query"])
    return {"documents": results}

# Route based on grade
def decide_generation(state):
    return (
        "generate"
        if state["grade"] == "relevant"
        else "web_search"
    )

workflow = StateGraph(GraphState)
workflow.add_node("retrieve", retrieve)
workflow.add_node("grade", grade_documents)
workflow.add_node("web_search", web_search)
workflow.add_conditional_edges(
    "grade", decide_generation
)
```



Adaptive RAG - Route by Query Type

📘 WHAT IT ADDS

A **query classifier** at the front that decides which strategy to use before retrieval even starts. Different questions need different approaches.

🏠 SCENARIO

HR assistant handles:

"What's our vacation policy?" → simple lookup

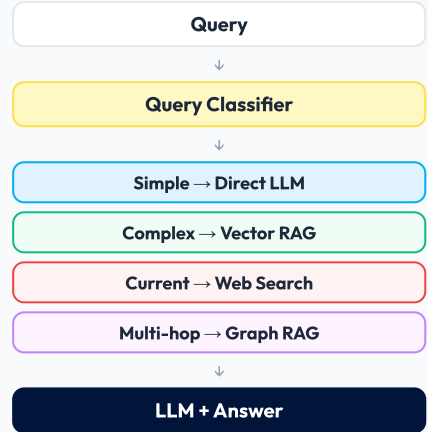
"Compare EU vs US leave policy" → multi-doc

"Today's office news?" → web search

✓ WHEN TO USE

- Mixed query types in same system
- Some questions need web, some need docs
- You want to save cost on simple queries

≡ PIPELINE



</> CLASSIFIER

```

type = classifier.predict(query)
# simple / complex /
# current / multi_hop
  
```



GraphRAG - Retrieve Relationships

📌 WHAT IT ADDS

Builds a **knowledge graph** on top of documents. Entities (people, orgs, concepts) become nodes. Relationships become edges. Retrieval traverses connections, not just similarity.

⚡ WHY NAIVE RAG FAILS HERE

Query: "How is CEO John connected to the acquisition?"

Naive RAG retrieves text about CEO and text about acquisition separately - misses the connection. GraphRAG traverses the entity graph and returns the relationship path.

✓ BEST FOR

- Knowledge bases with rich entity relationships
- Fraud detection, org charts, supply chain
- Multi-hop reasoning queries

🔗 GRAPH STRUCTURE

```
Documents ↓ NLP extraction [CEO
John] —LEADS→ [Company X] | |
SIGNED ACQUIRED | | ↓ ↓ [Contract
Y] ← [Company Z] | SUPPLIES | ↓
[Division A] Query: "How is John
connected to the acquisition?" →
traverse 2 hops → answer
```

✂ TOOLS

Microsoft GraphRAG · Neo4j + LangChain ·
NebulaGraph · Triplex



AI Coach John

FOLLOW



↻ Repost

GraphRAG with Neo4j

</> BUILD + QUERY A KNOWLEDGE GRAPH

```
from langchain_community.graphs import (
    Neo4jGraph
)
from langchain_experimental.graph_transformers\
    import LLMGraphTransformer

graph = Neo4jGraph(
    url="bolt://localhost:7687",
    username="neo4j",
    password="password"
)

# Step 1: Extract entities + relationships
transformer = LLMGraphTransformer(llm=llm)
graph_docs = transformer.convert_to_graph_documents(
    docs
)
graph.add_graph_documents(graph_docs)

# Step 2: Graph-based retrieval
from langchain.chains import GraphCypherQAChain

chain = GraphCypherQAChain.from_llm(
    llm=llm,
    graph=graph,
    verbose=True
)

# LLM auto-generates Cypher query
result = chain.invoke({
    "query": (
        "How is CEO John connected "
        "to the acquisition of Company Z?"
    )
})
```



Agentic RAG - Reason to Retrieve

📌 WHAT IT ADDS

The LLM doesn't just retrieve once. It **decides when to retrieve, what to retrieve, and whether the answer is good enough** - or if it needs to search again.

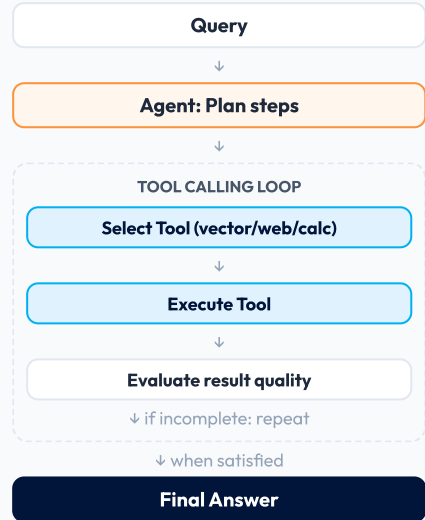
⚡ SCENARIO

Query: "Compare our Q3 revenue to competitors and recommend strategic adjustments."

Agent:

1. Retrieves internal Q3 report
2. Searches for competitor earnings
3. Calculates deltas
4. Evaluates if enough data
5. Retrieves strategy playbook
6. Synthesizes recommendation

≡ AGENTIC LOOP



✂️ FRAMEWORKS

LangGraph ReAct · LlamaIndex Agents · Anthropic Tool Use



AI Coach John

FOLLOW



↺ Repost

Agentic RAG with Anthropic Tool Use

</> REACT AGENT WITH RAG TOOL

```
from anthropic import Anthropic

client = Anthropic()

tools = [{
    "name": "retrieve_documents",
    "description": "Search internal docs",
    "input_schema": {
        "type": "object",
        "properties": {
            "query": {"type": "string"}
        },
        "required": ["query"]
    }
}, {
    "name": "web_search",
    "description": "Search the web",
    "input_schema": {
        "type": "object",
        "properties": {
            "query": {"type": "string"}
        },
        "required": ["query"]
    }
}]

def agentic_rag(user_query: str) -> str:
    messages = [
        {"role": "user", "content": user_query}
    ]

    while True:
        response = client.messages.create(
            model="claude-opus-4-8",
            max_tokens=4096,
            tools=tools,
            messages=messages
        )

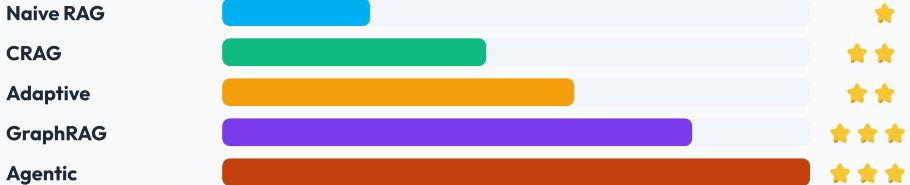
        if response.stop_reason == "end_turn":
            return response.content[0].text

        # Execute tool and continue loop
        for block in response.content:
            if block.type == "tool_use":
                result = execute_tool(
                    block.name, block.input
                )
                messages.append(
                    tool_result(block.id, result)
                )
```



Which Architecture Should You Use?

COMPLEXITY VS RETRIEVAL POWER



ARCHITECTURE	USE WHEN	AVOID WHEN
Naive RAG	MVP, consistent docs	High-stakes production
CRAG	Mixed quality corpus	All docs are reliable
Adaptive	Diverse query types	Only one query type
GraphRAG	Relationship queries	No entity structure
Agentic	Multi-step research	Real-time latency need



AI Coach John

FOLLOW



 Repost

The Evolution of RAG Architectures

1

NAIVE RAG (2022)

Simple embed-and-retrieve. Worked great for demos, failed at scale.

2

CRAG (2024 JAN)

Added retrieval quality grading. First to admit "I don't have reliable info - let me search."

3

ADAPTIVE RAG (2024)

Added query routing. Recognize the question type before choosing a retrieval strategy.

4

GRAPHRAG (2024 APR)

Microsoft open-sourced graph-based retrieval for entity-relationship queries. Game changer for structured knowledge.

5

AGENTIC RAG (2024-25)

LLM drives its own retrieval loop. Closest to how a human researcher actually works.

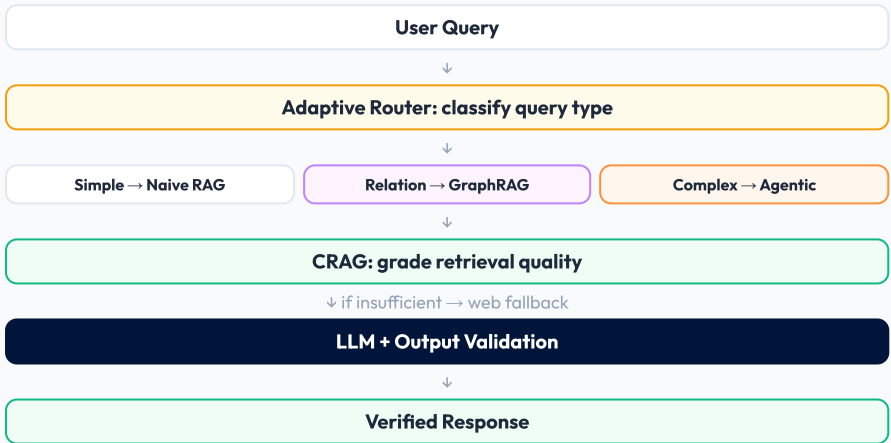
**AI Coach John****FOLLOW** **Repost**

Combine Architectures **for Real Production**

i THE REAL PATTERN

Most production systems are **hybrids** - they combine multiple architectures. Adaptive routing selects which RAG engine to use. CRAG validates the result. Agentic loop fills gaps.

👤 HYBRID ARCHITECTURE



RAG Architectures Cheatsheet

ARCHITECTURE	CORE FEATURE	LATENCY	COMPLEXITY
Naive RAG	Fixed-K vector search	Fast	Low
CRAG	Retrieval grading + fallback	Medium	Medium
Adaptive	Query-type routing	Fast-Medium	Medium
GraphRAG	Entity relationship traversal	Medium	High
Agentic	LLM-driven retrieval loop	Slow	High

🔗 **Start with Naive RAG. Add CRAG when quality drops. Add Adaptive when query types diverge. Add Graph when relationships matter. Add Agentic when tasks get complex.**

